JAVA OOP LABORATORY EXERCISE

CAMPUS CAFÉ ORDERING SYSTEM

Student Name: LAQUINTA, JANRELL Y.

Section: E

Subject: ITS 202

Date: NOVEMBER 9, 2026

Submitted as a laboratory exercise demonstrating Java object-oriented programming concepts.

1. Introduction

The Campus Café Ordering System is a Java console app that handles customer food and drink orders.

- **Takes Orders:** Customers can choose up to three items (a beverage, pastry, or meal) and select options for each.
- **Calculates Totals:** It figures out the price based on the selected items and options.
- **Applies Discounts:** Students automatically get a 10% discount.
- **Accepts Vouchers:** Customers can apply a voucher to lower their balance.
- **Rewards Loyalty:** It calculates and awards loyalty points based on the final amount paid.

2. OOP Concepts Used

- **Conditional Statements (`if` / `else`):** The program checks rules before making decisions, such as ensuring prices aren't negative, adding extra charges for warming pastries or upsizing drinks, enforcing the maximum three-item limit, and checking if the customer is a student for a discount.

- **Classes and Objects:** The app is split into seven distinct classes. The `CafeApp` class acts as the main runner, creating and managing `Customer`, `Beverage`, `Pastry`, `Meal`, and `Order` objects.

- **Object Interaction:** An `Order` object holds one `Customer` and up to three items. It talks to each item by calling its `computePrice()` method and updates the customer's loyalty points after checkout.

- **Constructors and Overloading:** Classes use special methods called constructors to set up initial values. Some classes have multiple versions (overloading) so they can be initialized with different details—for instance, a `Customer` can be created with just a name, with a name and student status, or with default settings.

- **Encapsulation (Data Hiding):** Sensitive variables are kept private or protected. The program uses getters and setters to access and change data safely, with setters rejecting invalid values like negative prices or invalid sizes.

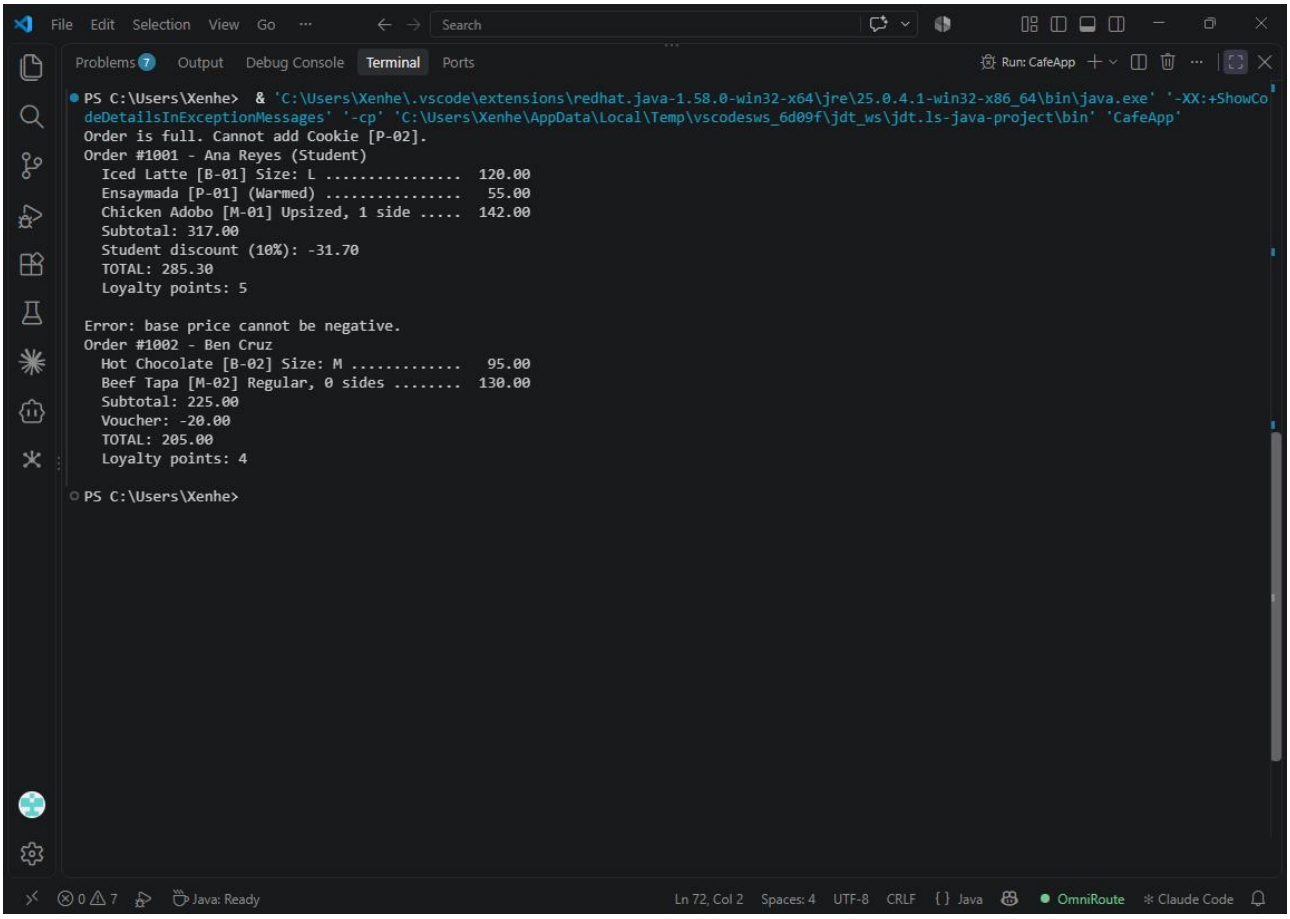
- **Method Overloading:** The program has methods with the same name that handle different types of data. For example, `Customer` can add loyalty points using either an integer or a decimal number, and `Order` can calculate the total with or without a voucher.

- **Method Overriding and Polymorphism:** Subclasses (`Beverage`, `Pastry`, and `Meal`) customize how `computePrice()` and `getDescription()` work. Because the `Order` treats them generically as `MenuItem` references, Java automatically runs the correct subclass version at runtime.

- **Abstraction:** `MenuItem` serves as a blueprint (an abstract class) that forces all food and drink items to have a `computePrice()` method, leaving the specific pricing rules up to each individual subclass.

4. Sample Program Output

This image was generated from the compiled program's actual console output.



5. Challenges Encountered and Solutions

Challenge 1: Managing item slots without collections. Because the use of ArrayList or any java.util collection was strictly prohibited, storing up to three items per order required explicit individual attribute tracking (item1, item2, item3) combined with an itemCount tracker. Solution: Implemented an addItem(MenuItem item) method using an if-else chain to sequentially assign items to the first available null slot, returning a warning message when itemCount reached three.

Challenge 2: Enforcing secure data encapsulation with validation. Ensuring that negative prices or invalid beverage sizes (S, M, L) were completely rejected required writing defensive setter methods. Solution: Added explicit validation checks inside setBasePrice(), setSize(), and setExtraSides() to intercept invalid inputs, print error messages when necessary, and preserve safe previous states.